

Safe Reinforcement Learning Using Black-Box Reachability Analysis

Mahmoud Selim¹, Amr Alanwar², Shreyas Kousik³, Grace Gao³, Marco Pavone³, and Karl H. Johansson⁴

Abstract—Reinforcement learning (RL) is capable of sophisticated motion planning and control for robots in uncertain environments. However, state-of-the-art deep RL approaches typically lack safety guarantees, especially when the robot and environment models are unknown. To justify widespread deployment, robots must respect safety constraints without sacrificing performance. Thus, we propose a Black-box Reachability-based Safety Layer (BRS�) with three main components: (1) data-driven reachability analysis for a black-box robot model, (2) a trajectory rollout planner that predicts future actions and observations using an ensemble of neural networks trained online, and (3) a differentiable polytope collision check between the reachable set and obstacles that enables correcting unsafe actions. In simulation, BRS� outperforms other state-of-the-art safe RL methods on a Turtlebot 3, a quadrotor, a trajectory-tracking point mass, and a hexarotor in wind with an unsafe set adjacent to the area of highest reward.

Index Terms—Reinforcement Learning, Robot Safety, Task and Motion Planning

I. INTRODUCTION

In reinforcement learning (RL), an agent perceives and reacts to consecutive states of its environment to maximize long-term cumulative expected reward [1]. One key challenge to the widespread deployment of RL in safety-critical systems is ensuring that an RL agent’s policies are safe, especially when the system environment or dynamics are a black box and subject to noise [2], [3]. In this work, we consider RL for guaranteed-safe navigation of mobile robots, such as autonomous cars or delivery drones, where safety means collision avoidance. We leverage RL to plan complex action sequences in concert with data-driven reachability analysis to guarantee safety for a black-box system.

A. Related Work

Safe RL aims to learn policies that maximize expected reward on a task while respecting safety constraints during

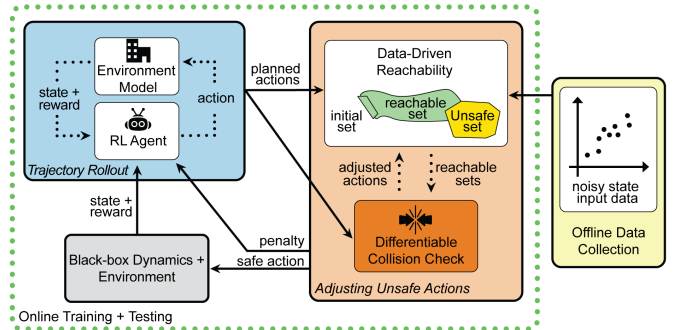


Fig. 1: Overview of the proposed BRS� method (link to video). Given data collected offline (in yellow, right), we perform online safe training and deployment of an RL agent. The RL agent creates trajectory plans for a robot in a receding-horizon way as follows. Each planning iteration is one clockwise loop in the green dashed box. First (blue, top left), the agent predicts a possible future trajectory by rolling out its current policy with an ensemble of neural networks trained online to model the black-box environment (grey, bottom left). Second (orange, middle), the candidate plan is *adjusted* to ensure safety using data-driven reachability and a constrained, differentiable method of collision-checking our robot’s reachable sets. We execute a failsafe maneuver if the collision check is infeasible. Finally, the new safe plan is passed to the robot, and a penalty is passed to the RL agent for choosing unsafe action.

both learning and deployment [3]. Existing methods can be roughly classified as *objective-based* or *exploration-based*, depending on how safety is formulated. We first discuss these categories, then the specific case of mobile robot navigation, which we use to evaluate our proposed method.

Objective-based methods encourage safety by penalizing constraint violations in the objective. This can be done by relating cumulative reward to the system’s risk, such as the probability of visiting error states [4]. In practice, this results in an RL agent attempting to minimize an empirical risk measure (that is, an approximation of the probability of entering a dangerous or undesired state). Similarly, one can penalize the probability of losing reward (by visiting an unsafe state) for a given action [2], in which case the agent minimizes temporal differences in the reward and thus also minimizes risk. Another approach is to restrict policies to be ergodic with high probability, meaning any state can eventually be reached from any other state [5]. This is a more general problem, which comes at a cost: feasible safe policies do not always exist, and the algorithms are far more complex. While these methods can make an agent prefer safe actions, they cannot guarantee safety during training or deployment. Another group of objective-based algorithms aims to modify the Markov Decision Process (MDP) that the RL agent tries to optimize. Some model safe optimization problems as maximizing an unknown expected reward function [6]. However, they exploit regularity assumptions on the function wherein similar decisions are associated with similar rewards. They also assume the bandit setting, where decisions do not cause state transitions. Others

Manuscript received: Feb 24, 2022; Revised: May 11, 2022; Accepted: June 12, 2022

This paper was recommended for publication by Editor Jens Kober upon evaluation of the Associate Editor and Reviewers’ comments

This work was supported by the Swedish Research Council, and the Knut and Alice Wallenberg Foundation. Toyota Research Institute provided funds to support this work. The NASA University Leadership initiative (grant #80NSSC20M0163) provided funds to assist the authors with their research, but this article solely reflects the opinions and conclusions of its authors and not any NASA entity.

¹Ain Shams University, Cairo, Egypt. ²Jacobs University, Bremen, Germany. ³Stanford University, Stanford, CA, USA. ⁴KTH Royal Institute of Technology, Stockholm, Sweden. Corresponding author: mahmoud.selim@eng.asu.edu.eg.

Digital Object Identifier (DOI): see top of this page

utilize constrained MDPs [7] to enforce safety in various RL settings, either online or offline. Online methods learn by coupling the iteration of numerical optimization algorithms (such as primal-dual gradient updates) with data collection [8], [9], [10], [11]. These algorithms have also been studied in exploration-based settings [12], [13]. However, they provide no guarantees on safety during training. On the other hand, offline schemes separate optimization and data collection [14], [15]. They conservatively enforce safety constraints on every policy iteration but are more challenging to scale up.

Exploration-based methods modify the agent's exploration process instead of its optimization criterion. Exploration is the process of learning about unexplored states by trying random actions or actions that are not expected to yield maximum reward (for example, an ϵ -greedy strategy). However, visiting unexplored states naively can harm a robot or its environment. To avoid this, one can aim to guarantee safety during both exploration and exploitation, in both training and testing, by modifying the exploration strategy to incorporate risk metrics [16]. One can also use prior knowledge as an inductive bias for the exploration process [3], [17]; for example, one can provide a finite set of demonstrations as guidance on the task [18]. Other approaches use control theory to guide or constrain the RL agent's actions. Most of these approaches use system models with Lyapunov or control barrier functions (CBFs) to guarantee safety (or stability) of the system [19], [20], [21]. One can also combine data-driven approaches with model-free barrier or intervention functions [22], [23], [24], or use robust CBFs to model uncertain parts of a system [25], [26]. Although these approaches can provide strong guarantees, most assume the system is control-affine, and need prior knowledge on some or all of the system model [27], [28], [29], which may not always be feasible. Finally, our method is most similar to [30], which learns a safety signal from data offline, then uses it to adjust an RL agent's controls at runtime. This method uses a first-order safety approximation for fast adjustment, but (as we show) can be conservative.

Safe navigation is a fundamental challenge in robotics, because robots typically have uncertain, nonlinear dynamics. Classical techniques such as A* or RRT [31, Ch. 5] have been proposed to solve the navigation problem without learning. With these methods, safety has been enforced at different levels of the planning hierarchy, such as trajectory planning [32], or low-level control [33]. More recently, however, learning-based methods have been proposed [27], [28], [29]. Some safe RL navigation approaches depend on learning a value function of the expected time that an agent takes to reach the desired goal [34], [35]. Other approaches depend on learning the actions of the robot in an end-to-end manner [36], [37], [38], meaning that the agent attempts to convert raw sensor inputs (e.g., camera or LIDAR) into actuator commands. The key advantage of RL over traditional planners is in accelerating computation time and solution quality.

B. Proposed Method and Contributions

We propose a Black-box Reachability-based Safety Layer (BRS�), illustrated in Fig. 1, to enable strict safety guarantees for RL in an entirely data-driven way, addressing the above

challenges of lacking robot and environment models *a priori* and of enforcing safety for uncertain systems. We note that the main advantage of BRS� is the use of data-driven methods to provide safety guarantees for black-box system dynamics. BRS� enforces safety by computing a system's forward reachable set, which is the union of all trajectories that the system can realize within a finite or infinite time when starting from a bounded set of initial states, subject to a set of possible input signals [39]. Then, if the reachable set does not intersect with unsafe sets, the system is verified as safe, following similar arguments as in [32], [33], [40].

Limitations. Our method requires an approximation for the upper bound of a system's Lipschitz constant, similar to [29], [41], [42]. This results in a curse of dimensionality with respect to number of samples required to approximate the constant; note other sampling-based approaches scale similarly [29], [28]. Furthermore, we focus on a discrete-time setting, assume our robot can brake to a stop, and assume accurate perception of the robot's surroundings. We leave continuous-time (which can be addressed with similar reachability methods to ours [28], [40]) and perception uncertainty to future work.

Contributions. We show the following with BRS�:

- 1) We propose a safety layer by integrating data-driven reachability analysis with a differentiable polytope collision check and a trajectory rollout planner.
- 2) We demonstrate BRS� on robot navigation, where it outperforms a baseline RL agent, Reachability-based Trajectory Safeguard (RTS) [28], Safe Advantage-based Intervention for Learning policies with Reinforcement (SAILR) [24], and Safe Exploration in Continuous Action Spaces (SECAS) [30]. Our code is [online](#).

Next, in Section II, we provide preliminaries and formulate our safe RL problem. Sections III and IV discuss and evaluate the proposed approach. Finally, Section V presents concluding remarks and discusses future work.

II. PRELIMINARIES AND PROBLEM FORMULATION

This section presents the notation, set representations, system dynamics, and reachable set definitions used in this work. We then pose our safe RL problem.

A. Notation and Set Representations

The n -dimensional real numbers are $\mathbb{R}^n$, the natural numbers are $\mathbb{N}$, and the integers from n to m are $n:m$. We denote the element at row i and column j of matrix $\mathbf{A}$ by $(\mathbf{A})_{i,j}$, column j of $\mathbf{A}$ by $(\mathbf{A})_{:,j}$, and the element i of vector $\mathbf{a}$ by $(\mathbf{a})_i$. An $n \times m$ matrix of ones is $\mathbf{1}_{n \times m}$. For $\mathbf{A} \in \mathbb{R}^{n \times m}$ and $\mathbf{x} \in \mathbb{R}^n$, we use the shorthand $\mathbf{A} - \mathbf{x} = \mathbf{A} - \mathbf{x}\mathbf{1}_{1 \times m}$. The $\text{diag}(\cdot)$ operator places its arguments block-diagonally in a matrix of zeros. For a pair of sets A and B , the Minkowski sum is $A + B = \{\mathbf{a} + \mathbf{b} \mid \mathbf{a} \in A, \mathbf{b} \in B\}$, and the Cartesian product is $A \times B = \{(\mathbf{a}, \mathbf{b}) \mid \mathbf{a} \in A, \mathbf{b} \in B\}$.

We represent sets using constrained zonotopes, zonotopes, and intervals, because they enable efficient Minkowski sum computation (a key part of reachability analysis) [40] and collision checking via linear programming (critical to safe motion planning) [43]. A *constrained zonotope* [43] is a convex set parameterized by a center $\mathbf{c} \in \mathbb{R}^n$, generator matrix

$\mathbf{G} \in \mathbb{R}^{n \times n_g}$, constraint matrix $\mathbf{A} \in \mathbb{R}^{n_c \times n_g}$, and constraint vector $\mathbf{b} \in \mathbb{R}^{n_c}$ as

$$\mathcal{Z}(\mathbf{c}, \mathbf{G}, \mathbf{A}, \mathbf{b}) = \{\mathbf{c} + \mathbf{G}\mathbf{z} \mid \mathbf{A}\mathbf{z} = \mathbf{b}, \|\mathbf{z}\|_\infty \leq 1\}. \quad (1)$$

By [43, Thm. 1], every convex, compact polytope is a constrained zonotope and vice-versa. For polytopes represented as an intersection of halfplanes, we convert them to constrained zonotopes by finding a bounding box, then applying the halfspace intersection property in [44].

A zonotope is a special case of a constrained zonotope without equality constraints (but with $\|\mathbf{z}\|_\infty \leq 1$), which we denote $\mathcal{Z}(\mathbf{c}, \mathbf{G})$. For $Z = \mathcal{Z}(\mathbf{c}, \mathbf{G}) \subset \mathbb{R}^n$ and a linear map L , we have $LZ = \mathcal{Z}(L\mathbf{c}, L\mathbf{G})$; we denote $-Z = -1Z$. The Minkowski sum of two zonotopes $Z_1 = \mathcal{Z}(\mathbf{c}_1, \mathbf{G}_1)$ and $Z_2 = \mathcal{Z}(\mathbf{c}_2, \mathbf{G}_2)$ is given by $Z_1 + Z_2 = \mathcal{Z}(\mathbf{c}_1 + \mathbf{c}_2, [\mathbf{G}_1, \mathbf{G}_2])$ [40]. For an n -dimensional interval with lower (resp. upper) bounds $\underline{\mathbf{l}} \in \mathbb{R}^n$ (resp. $\bar{\mathbf{l}}$), we abuse notation to represent it as a zonotope $Z = \mathcal{Z}(\underline{\mathbf{l}}, \bar{\mathbf{l}}) \subset \mathbb{R}^n$, with center $\frac{1}{2}(\underline{\mathbf{l}} + \bar{\mathbf{l}})$ and generator matrix $\text{diag}(\frac{1}{2}(\bar{\mathbf{l}} - \underline{\mathbf{l}}))$.

B. Robot and Environment

We assume the robot can be described as a discrete-time, nonlinear control system with state $\mathbf{x}_k \in X \subset \mathbb{R}^n$ at time $k \in \mathbb{N}$. We assume the state space X is compact. The input $\mathbf{u}_k$ is drawn from a zonotope $U_k \subseteq U$ at each time k , where $U \subset \mathbb{R}^m$ is a zonotope of all possible actions. We denote process noise by $\mathbf{w}_k \in W \subset \mathbb{R}^n$, where W is specified later in Assumption 2. Finally, we denote the black box (i.e., unknown) dynamics $\mathbf{f} : X \times U \times W \rightarrow X$, for which

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k. \quad (2)$$

We further assume that $\mathbf{f}$ is twice differentiable and Lipschitz continuous, meaning there exists a *Lipschitz constant* L^* such that, if $\forall \mathbf{z}_1, \mathbf{z}_2 \in \mathbb{R}^{n+m}$ with $\mathbf{z}_j = (\mathbf{x}_j, \mathbf{u}_j)$, then $\|\mathbf{f}(\mathbf{z}_1) - \mathbf{f}(\mathbf{z}_2)\| \leq L^*\|\mathbf{z}_1 - \mathbf{z}_2\|$. We denote the initial state of the system as $\mathbf{x}_0$, drawn from a compact set $X_0 \subset \mathbb{R}^n$. Note that this formulation leads to an MDP.

To enable safety guarantees, we leverage the notion of failsafe maneuvers from mobile robotics [32], [45].

Assumption 1. *We assume the dynamics $\mathbf{f}$ are invariant to translation in position, and the robot can brake to a stop in $n_{\text{brk}} \in \mathbb{N}$ time steps and stay stopped indefinitely. That is, there exists $\mathbf{u}_{\text{brk}} \in U$ such that, if the robot is stopped at state $\mathbf{x}_k$, and if $\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_{\text{brk}})$, then $\mathbf{x}_{k+1} = \mathbf{x}_k$.*

Note, many real robots have a braking safety controller available, similar to the notion of an invariant set [29], [28]. Also, failsafe maneuvers exist even when a robot cannot remain stationary, such loiter circles for aircraft [46], [47].

We require that process noise obeys the following assumption for numerical tractability and robustness guarantees.

Assumption 2. *Each $\mathbf{w}_k$ is drawn uniformly from a noise zonotope $W = \mathcal{Z}(\mathbf{c}_w, \mathbf{G}_w)$ with $n_{g,w}$ generators.*

This formulation does not handle discontinuous changes in noise. However, there exist zonotope-based techniques to identify a change in W [48], after which one can compute the system's reachable set as in the present work. We leave measurement noise and perception uncertainty to future work.

We also note, in the case of Gaussian or unbounded noise, one can overapproximate a confidence level set of a probability distribution using a zonotope [40], [48].

We denote unsafe regions of state space, or *obstacles*, as $X_{\text{obs}} \subset X$. We assume obstacles are static but different in each episode, as the focus of this work is not on predicting other agents' motion. Furthermore, reachability-based frameworks exist to handle other agents' motion [33], [49], so the present work can extend to dynamic environments.

We further assume the robot can instantaneously sense all obstacles (that is, X_{obs}) and represent them as a union of constrained zonotopes. In the case of sensing limits, one can determine a minimum distance within which obstacles must be detected to ensure safety, given a robot's maximum speed and braking distance [32, Section 5].

C. Reachable Sets

We ensure safety by computing our robot's forward reachable set (FRS) for a given motion plan, then adjusting the plan so that the FRS lies outside of obstacles. We define the FRS, henceforth called the reachable set, as follows:

Definition 1. *The reachable set R_k at time step k , subject to a sequence of inputs $\mathbf{u}_j \in U_j \subset \mathbb{R}^m$, noise $\mathbf{w}_j \in W \forall j \in \{0, \dots, k-1\}$, and initial set $X_0 \in \mathbb{R}^n$, is the set*

$$R_k = \{\mathbf{x}_k \in \mathbb{R}^n \mid \mathbf{x}_{j+1} = \mathbf{f}(\mathbf{x}_j, \mathbf{u}_j) + \mathbf{w}_j, \mathbf{x}_0 \in X_0, \mathbf{u}_j \in U_j, \text{ and } \mathbf{w}_j \in W, \forall j = 0, \dots, k-1\}. \quad (3)$$

Recall that we treat the dynamics $\mathbf{f}$ as a black box (e.g., a simulator), which could be nonlinear and difficult to model, but we still seek to conservatively approximate (that is, overapproximate) the reachable set R_k .

D. Safe RL Problem Formulation

We denote the state of the RL agent at time k by $\hat{\mathbf{x}}_k \in \mathbb{R}^{n_{\text{RL}}}$, which contains the state $\mathbf{x}_k$ of the robot plus information such as sensor measurements and previous actions. At each time k , the RL agent chooses $\mathbf{u}_k$. Recall that $X_{\text{obs}} \subset \mathbb{R}^n$ denotes obstacles. For a given task, we construct a reward function $\rho : (\hat{\mathbf{x}}_k, \mathbf{u}_k) \mapsto r_k \in \mathbb{R}$ (examples of ρ are given in Section IV). At time k , let $\mathbf{p}_k = (\mathbf{u}_j)_{j=k}^{n_{\text{plan}}}$ denote a *plan*, or sequence of actions, of duration $n_{\text{plan}} \in \mathbb{N}$.

Then, our safe RL problem is as follows. We seek to learn a policy $\pi_\theta : \hat{\mathbf{x}}_k \mapsto \mathbf{u}_k$, represented by a neural network with parameters θ , that maximizes expected cumulative reward. Note that the policy can be deterministic or stochastic. Since rolling out the policy naively may lead to collisions, we also seek to create a safety layer between the policy and the robot (that is, to ensure $R_j \cap X_{\text{obs}} = \emptyset$ for all $j \geq k$).

III. BLACK-BOX REACHABILITY-BASED SAFETY LAYER

We unite three components into our BRSL system for collision-free motion planning without a dynamic model of the robot or its surroundings *a priori*. The first component is an environment model, learned online. To find high reward actions (i.e., motion plans) for this model, the second component is an RL agent. Since the agent may create unsafe plans, our third component is a safety layer that combines data-driven reachability analysis with differentiable collision checking to

Algorithm 1: Safe RL with BRSL

```

1 initialize the RL agent with a random policy  $\pi_\theta$ ,
  environment model  $\mu_\phi$ , empty replay buffer  $B$ , max
  number of time steps  $n_{\text{iter}}$ , and a safe plan  $\mathbf{p}_0$ 
2 for each episode do
3   initialize task with reward function  $\rho$ 
4    $\hat{\mathbf{x}}_1 \leftarrow$  observe initial environment state
5   for  $k = 1 : n_{\text{iter}}$  do
6      $\mathbf{p}_k \leftarrow$  roll out a trajectory
7      $\hat{R}_k \leftarrow \mathcal{Z}(\mathbf{x}_k, \mathbf{0})$  // init. reachable set
8      $(\hat{R}_j)_{j=k}^{k+n_{\text{plan}}} \leftarrow \text{reach}(\hat{R}_k, \mathbf{p}_k)$  // use Alg. 2
9     if any  $\hat{R}_j \cap X_{\text{obs}} \neq \emptyset$  then
10      try  $\mathbf{p}_k \leftarrow \text{adjust}(\mathbf{p}_k, X_{\text{obs}})$  // use Alg. 3
11      catch execute failsafe maneuver; continue
12      $\mathbf{u}_k \leftarrow$  get first (safe) action from  $\mathbf{p}_k$ 
13      $r_k \leftarrow \rho(\hat{\mathbf{x}}_k, \mathbf{u}_k)$  // get reward
14      $\hat{\mathbf{x}}_{k+1} \leftarrow$  observe next environment state
15     add  $(\hat{\mathbf{x}}_k, \mathbf{u}_k, r_k, \hat{\mathbf{x}}_{k+1})$  to  $B$ 
16     train the RL agent  $\pi_\theta$  and the environment
        model  $\mu_\phi$  using minibatch from  $B$ 

```

enable safe trajectory optimization. Theorem 1 summarizes safety via BRSL.

BRSL is summarized in Algorithm 1. It uses a receding-horizon strategy to create a new safe plan $\mathbf{p}_k$ in each k^{th} receding-horizon motion planning iteration. Consider a single planning iteration (that is, time step k) (Lines 4–16). Suppose the RL agent has previously created a safe plan $\mathbf{p}_{k-1}$ (such as staying stopped indefinitely). At the beginning of the iteration, BRSL creates a new plan $\mathbf{p}_k$ by rolling out the RL agent along with an environment model. Next, BRSL chooses a safe action by adjusting the rolled-out action sequence (Lines 9–11) such that the corresponding reachable set (computed with Algorithm 2) is collision-free and ends with a failsafe maneuver. If the adjustment procedure (as in Algorithm 3) fails to find a safe plan, then the robot executes the failsafe maneuver. Finally, BRSL sends the first action in the current safe plan to the robot, gets a reward, and trains the RL agent and environment model (Lines 12–16). To enable training our environment model online, we collect data in a replay buffer B at each time k (Line 15). We note that BRSL can be used during both training and deployment. That is, the safety layer can operate even for an untrained policy. Thus, for training, we initialize π_θ with random weights.

To proceed, we detail our methods for data-driven reachability and adjusting unsafe actions.

A. Data-Driven Reachability Analysis

BRSL performs data-driven reachability analysis of a plan $\mathbf{p}_k = (\mathbf{u}_j)_{j=k}^{k+n_{\text{plan}}}$ using Algorithm 2, based on [42]. Algorithm 2 overapproximates the reachable set as in (3) by computing a zonotope $\hat{R}_j \supseteq R_j$ for each time step of the current plan.

Our reachability analysis uses noisy trajectory data of the black-box system model collected offline; we use data collected online only for training the policy and environment

Algorithm 2: Black-box System Reachability [42]

```

Input: initial reachable set  $\hat{R}_0$ , actions  $(\mathbf{u}_j)_{j=k}^{k+n_{\text{plan}}}$ 
Parameter: state/action data  $(\mathbf{X}_-, \mathbf{X}_+, \mathbf{U}_-)$ , noise
  zonotope  $W = \mathcal{Z}(\mathbf{c}_w, \mathbf{G}_w)$ , Lipschitz
  constant  $L^*$ , covering radius  $\delta$ 
1  $Z_\epsilon \leftarrow \mathcal{Z}(\mathbf{0}, \text{diag}((L^*)_1(\delta)_1/2, \dots, (L^*)_n(\delta)_n/2))$  for
   $j = k : (k + n_{\text{plan}})$  do
2    $\mathbf{M}_j \leftarrow (\mathbf{X}_+ - \mathbf{c}_w) \begin{bmatrix} \mathbf{1}_{1 \times t_{\text{total}}} \\ \mathbf{X}_- - \mathbf{x}_j^* \\ \mathbf{U}_- - \mathbf{u}_j \end{bmatrix}^\dagger$ 
3    $\mathbf{l} \leftarrow \min_j \left( (\mathbf{X}_+)_{:,j} - \mathbf{M}_j \begin{bmatrix} 1 \\ (\mathbf{X}_-)_{:,j} - \mathbf{x}_j^* \\ (\mathbf{U}_-)_{:,j} - \mathbf{u}_j \end{bmatrix} \right)$ 
4    $\bar{\mathbf{l}} \leftarrow$  same as  $\mathbf{l}$ , but use max instead of min
5    $Z_L \leftarrow \mathcal{Z}(\mathbf{l}, \bar{\mathbf{l}}) - W$  and  $U_j \leftarrow \mathcal{Z}(\mathbf{u}_j, \mathbf{0})$ 
6    $\hat{R}_{j+1} \leftarrow \mathbf{M}_j (1 \times (\hat{R}_j - \mathbf{x}_j^*) \times (U_j - \mathbf{u}_j)) + W + Z_L + Z_\epsilon$ 
7 return  $(\hat{R}_j)_{j=k}^{k+n_{\text{plan}}}$  // overapproximates (3)

```

model. We consider q input-state trajectories of lengths $t_i \in \mathbb{N}$, $i = 1, \dots, q$, with total duration $t_{\text{total}} = \sum_i^q t_i$. We denote the data as $(\mathbf{x}_k^{(i)})_{k=0}^{t_i-1}$, $(\mathbf{u}_k^{(i)})_{k=0}^{t_i-1}$, $i = 1, \dots, q$. To ease notation for the various matrix operations needed in Algorithm 2, we collect the data in matrices:

$$\mathbf{X}_- = [\mathbf{x}_0^{(1)}, \dots, \mathbf{x}_{t_1-1}^{(1)}, \mathbf{x}_0^{(2)}, \dots, \mathbf{x}_0^{(q)}, \dots, \mathbf{x}_{t_q-1}^{(q)}], \quad (4a)$$

$$\mathbf{X}_+ = [\mathbf{x}_1^{(1)}, \dots, \mathbf{x}_{t_1}^{(1)}, \mathbf{x}_1^{(2)}, \dots, \mathbf{x}_1^{(q)}, \dots, \mathbf{x}_{t_q}^{(q)}], \quad (4b)$$

$$\mathbf{U}_- = [\mathbf{u}_0^{(1)}, \dots, \mathbf{u}_{t_1-1}^{(1)}, \mathbf{u}_0^{(2)}, \dots, \mathbf{u}_0^{(q)}, \dots, \mathbf{u}_{t_q-1}^{(q)}]. \quad (4c)$$

Note, the time steps are different in $\mathbf{X}_-$ and $\mathbf{X}_+$ to simplify considering state transitions corresponding to the actions in $\mathbf{U}_-$. Selecting enough data to sufficiently capture system behavior is a challenge that depends on the system, though specific sampling strategies exist for some systems [28].

We must approximate the Lipschitz constant of the dynamics for our reachability analysis, which we do from the data $(\mathbf{X}_-, \mathbf{X}_+, \mathbf{U}_-)$ with the method in [42, Section 4, Remark 1]. We also require a data covering radius δ such that, for any data point $\mathbf{z}_1 \in X \times U$, there exists another data point $\mathbf{z}_2 \in X \times U$ for which $\|\mathbf{z}_1 - \mathbf{z}_2\|_2 \leq \delta$. We assume sufficiently many data points are known *a priori* to upper-bound L^* and lower-bound δ ; and, we assume L^* and δ are the same for offline data collection and online operation. Note, prior work assumes similar bounds [41], [42].

We find that the reachable set becomes conservative (i.e., large) if the same L^* and δ are used for every dimension, because the true dynamics are typically scaled differently in each state dimension. To mitigate this source of conservativeness, we approximate a different $(L^*)_i$ and $(\delta)_i$ for each dimension, which we then use to compute a Lipschitz zonotope Z_ϵ (see Line 1 of Algorithm 2). Note, this is an improvement over prior work [42].

B. Adjusting Unsafe Actions

After the RL agent rolls out a plan $\mathbf{p}_k$, the safety layer adjusts it to ensure it is safe. This is done by checking the

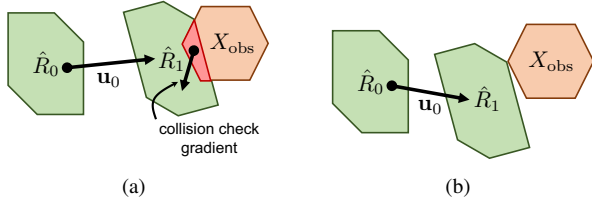


Fig. 2: We move zonotope reachable sets out of intersection (see Alg. 3) by using the gradient of a collision check, shown in (a), to adjust $\mathbf{u}_0$ so the reachable sets are out of collision as shown in (b).

intersection of the plan's reachable sets with unsafe sets. Note, our proposed adjustment procedure does not depend on π_θ , only on the unsafe sets around the robot. The plan is applied to the environment if all of its actions are safe; otherwise, we search for a safe plan. One strategy for finding a safe plan is to sample randomly in the action space [28], but this can be prohibitively expensive in the large action spaces that arise from choosing control inputs at multiple time steps. Instead, we use gradient descent to adjust our plan such that the reachable sets are not in collision, and such that the plan has a failsafe maneuver.

We adjust unsafe actions using Algorithm 3. If the algorithm does not complete within the duration of one time step (in other words, we fix the rate of receding-horizon planning), we terminate it and continue our previously-found safe plan. Our method steps through each action in a plan $\mathbf{p}$ and performs the following. First, we compute the reachable set for all remaining time steps with Algorithm 2 (Line 5). Second, we collision check the reachable set (Line 6) as detailed below. Third, if the reachable sets are in a collision, we compute the gradient of the collision check and perform projected gradient descent (Line 8) as in fig 2. Finally, if the algorithm converges to a safe plan, we return it, or else return “unsafe.” Note, the final plan must have a failsafe maneuver (Line 11).

We collision check reachable and unsafe sets, all represented as constrained zonotopes, as follows. Consider two constrained zonotopes, $Z_1 = \mathcal{Z}(\mathbf{c}_1, \mathbf{G}_1, \mathbf{A}_1, \mathbf{b}_1)$ and $Z_2 = \mathcal{Z}(\mathbf{c}_2, \mathbf{G}_2, \mathbf{A}_2, \mathbf{b}_2)$. Applying [43, Prop. 1], their intersection is $Z_\cap = Z_1 \cap Z_2 = \mathcal{Z}(\mathbf{c}_\cap, \mathbf{G}_\cap, \mathbf{A}_\cap, \mathbf{b}_\cap)$, given by

$$Z_\cap = \mathcal{Z}\left(\mathbf{c}_1, [\mathbf{G}_1, \mathbf{0}], \begin{bmatrix} \mathbf{A}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_2 \end{bmatrix}, \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}\right). \quad (5)$$

We check if $Z_1 \cap Z_2$ is empty by solving a linear program, as per [43, Prop. 2]:

$$v^* = \min_{\mathbf{z}, v} \{v \mid \mathbf{A}_\cap \mathbf{z} = \mathbf{b}_\cap \text{ and } |\mathbf{z}| \leq v\}, \quad (6)$$

with $|\mathbf{z}|$ taken elementwise; $Z_\cap$ is nonempty iff $v \leq 1$. Note, (6) is feasible when Z_1 and Z_2 have feasible constraints.

We use gradient descent to move our reachable sets $\hat{R}_k$ out of collision. Since we use (6) for collision checking, we differentiate its solution with respect to the problem parameters using [50], [51]. Let $\hat{\mathbf{c}}_k$ denote the center of $\hat{R}_k$. Per Algorithm 2, $\hat{R}_k$ is a function of $\mathbf{u}_0, \dots, \mathbf{u}_{k-1}$. Let $(\mathbf{z}^*, v^*)$ be an optimal solution to (6) when the problem parameters (i.e., the input constrained zonotopes) are $\hat{R}_k$ and an unsafe set. Collision avoidance requires $v^* > 1$ [43, Prop. 2]. We compute the gradient $\nabla_{\mathbf{u}_k} v^*$ with respect to the input action (assuming a

Algorithm 3: Adjusting Unsafe Actions

Input: plan $\mathbf{p}_k = (\mathbf{u}_j)_{j=k}^{n_{\text{plan}}}$, obstacles X_{obs} , initial reachable set $\hat{R}_k$, step size γ , time limit t_{max} , time steps required to stop n_{brk}

```

1 // note  $\mathbf{p}_k$  has failsafe  $\mathbf{u}_{\text{brk}}$  for all  $j > k + n_{\text{plan}}$ 
2  $\mathbf{p}_{\text{safe}} \leftarrow \mathbf{p}_k$  // initialize with given plan
3 for  $j = k : (k + n_{\text{plan}} + n_{\text{brk}})$  do
4   while time limit not exceeded do
5      $(\hat{R}_j)_{j=k}^{k+n_{\text{plan}}} \leftarrow \text{reach}(\hat{R}_j, \mathbf{p}_{\text{safe}})$  // use Alg. 2
6      $v^* \leftarrow \text{collision check } \hat{R}_j \cap X_{\text{obs}}$  using (6)
7     if  $v^* \leq 1$  (i.e., in collision) then
8        $\mathbf{u}_j \leftarrow \text{proj}_{U_j}(\mathbf{u}_j + \gamma \nabla_{\mathbf{u}_j} v^*)$  // using (7)
9     else
10      break and restart inner while loop
11 if all  $\hat{R}_j \cap X_{\text{obs}} = \emptyset$  and  $\mathbf{x}_n$  is stopped then
12   return  $\mathbf{p}_{\text{safe}} = (\mathbf{u}_j)_{j=k}^{n_{\text{plan}}}$  // found new safe plan
13 else
14   return error “unsafe” // failed to find safe plan
```

constant linearization point) using a chain rule recursion with $i = 0, \dots, n_{\text{plan}}$ given by

$$\nabla_{\mathbf{u}_h} v^* = \nabla_{\hat{\mathbf{c}}_k} v^* \nabla_{\hat{\mathbf{c}}_{k-1}} \hat{\mathbf{c}}_k \left(\prod_{j=h+2}^{j=k-1} \nabla_{\hat{\mathbf{c}}_{j-1}} \hat{\mathbf{c}}_j \right) \nabla_{\mathbf{u}_h} \hat{\mathbf{c}}_{h+1}, \quad (7)$$

with $h = k - i$. The gradients of $\hat{\mathbf{c}}_k$ are given by

$$\nabla_{\hat{\mathbf{c}}_{k-1}} \hat{\mathbf{c}}_k = (\mathbf{M}_{k-1})_{(1:1+n), (1:1+n)}, \text{ and} \quad (8a)$$

$$\nabla_{\mathbf{u}_{k-1}} \hat{\mathbf{c}}_k = (\mathbf{M}_{k-1})_{:, (n+1:n+1+m)}, \quad (8b)$$

where $\mathbf{M}_{k-1}$ is computed as in Algorithm 2, Line 2, and n and m are the state and action dimensions. After using $\nabla_{\mathbf{u}_k} v^*$ for gradient descent on $\mathbf{u}_k$, we project $\mathbf{u}_k$ to the set of feasible controls: $\text{proj}_{U_k}(\mathbf{u}_k) = \arg \min_{\mathbf{v} \in U_k} \{\|\mathbf{u}_k - \mathbf{v}\|_2^2\}$. The resulting controls may be unsafe, so we collision-check the final reachable sets at the end of Algorithm 3.

C. Analyzing Safety

We conclude this section by formalizing the notion that BRSRL enables safe RL.

Theorem 1. Suppose the assumptions on the robot and environment from Section II all hold, and, at time $k = 0$, the robot is at safe state. Suppose also that, at each time $k > 0$, the robot rolls out a new $\mathbf{p}_k$, then adjusts the plan using Algorithm 3. Then, the robot is guaranteed to be safe at all times $k \geq 0$.

Proof. We prove the claim by induction on k . At time 0, the robot can apply $\mathbf{u}_{\text{brk}}$ to stay safe for all time. Assume a safe plan exists at time $k \in \mathbb{N}$. Then, if the output of Algorithm 3 is unsafe (no new plan found), the robot can continue its previous safe plan; otherwise, if a new plan is found, the plan is safe for three reasons. First, the black-box reachability in Algorithm 2 is guaranteed to contain the true reachable set of the system [42, Theorem 2], because process noise is bounded by a zonotope as in Assumption 2. Second, when adjusting an

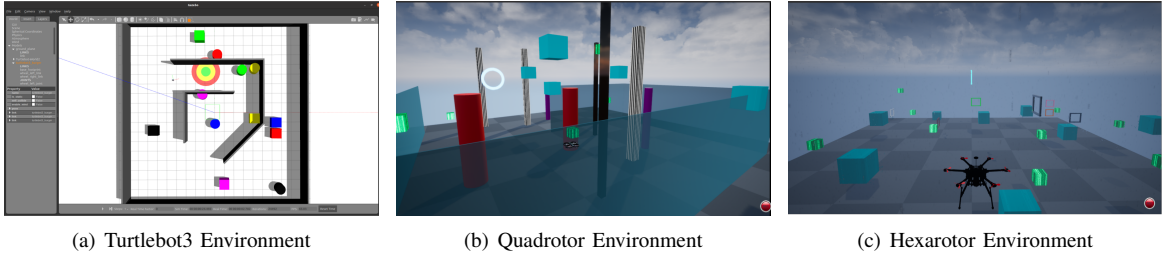


Fig. 3: Evaluation Environments.

unsafe plan with Algorithm 3, the zonotope collision check is guaranteed to always detect collisions [43, Prop. 2] to assess if $\hat{R}_j \cap X_{\text{obs}}$ is empty for each time step j of the plan. Third, Algorithm 3 requires that, after n_{plan} timesteps, the robot is stopped, so the new plan contains a failsafe maneuver, and the robot can safely apply $\mathbf{u}_{\text{brk}}$ for all time $j \geq k + n_{\text{plan}}$. $\square$

We note that the accuracy of the environment model, trained online, does not affect safety; Theorem 1 holds as long as the offline data are representative of the robot’s dynamics at runtime. We leave updating the data online for future work.

IV. EVALUATION

We evaluate BRSL on two types of environments: safe navigation to a goal (a Turtlebot in Gazebo and a quadrotor platform in Unreal Engine 4), and path following (a point mass based on [14], [24] and a hexarotor in wind in Unreal Engine 4). Figure 3 shows example environments. All code is run on a desktop computer with an Intel i5 11600 CPU and a RTX 3060 GPU. Our code is available [online](#). We aim to assess the following: (a) How does BRSL compare against other safe RL methods (RTS [28], SAILR [24], and SECAS [30])? (b) How conservative is BRSL? (c) Can BRSL run in real time?

Setup. We use TD3 [52] as our RL agent, after determining empirically that it outperforms SAC [53] and DDPG [54]. We randomly initialize the policy. Since the agent outputs continuous actions, to aid the exploration process, we inject zero-mean Gaussian noise with a variance of 0.5 that is dampened each time step. Note that this does not affect safety since our safety layer adjusts the output of the RL agent.

For each robot, to perform reachability analysis with Algorithm 2, we collect 500 time steps of noisy state/input data (as per (4)) offline in an empty environment while applying random control inputs. We found this quantity of data sufficient to ensure safety empirically; we leave a formal analysis of the minimum amount of data for future work.

We parameterize the environment model as an ensemble of neural networks, each modeling a Gaussian distribution over future states and observations. Each network has 4 layers, with hidden layers of size 200, and leaky ReLU activations with a negative slope of 0.01. We use a stochastic model wherein the ensemble predicts the parameters of a probability distribution, which is sampled to produce a state as in [55].

Goal-Based Environments. The Turtlebot 3 and the quadrotor seek to navigate to a random circular goal region $X_{\text{goal}} \subset X$ while avoiding randomly-generated obstacles $X_{\text{obs}} \subset X$. Each robot starts in a safe location at the center of the map. Each task is episodic, ending if the robot reaches the goal,

crashes, or exceeds a time limit. Both robots have uncertain, noisy dynamics as in (2). We discretize time at 10 Hz.

The Turtlebot’s control inputs are longitudinal velocity in $[0.00, 0.25]$ m/s and angular velocity in $[-0.5, 0.5]$ rad/s (these are the bounds of U_k). The robot has wheel encoders, plus a planar lidar that generates 18 range measurements evenly spaced in a 180° arc in front of the robot. The robot requires $n_{\text{brk}} = 6$ time steps to stop, so we set $n_{\text{plan}} = 8$.

The quadrotor control inputs are commanded velocities up to 5 m/s in each spatial direction at each time step. We note that we also experimented with learning low-level rotor speeds versus high-level velocity commands, and found that the velocity commands created the fairest testing conditions across all agents. The robot is equipped with an IMU and a 16-channel lidar which receives range measurements around the robot in a 50° vertical arc and a 360° horizontal arc. The robot has $n_{\text{brk}} = 10$, so we set $n_{\text{plan}} = 11$.

Path Following Environments. These experiments assess BRSL’s conservativeness by placing the highest reward adjacent to obstacles.

The goal for the point robot is to follow a circular path of radius r as quickly as possible while constrained to a region smaller than the target circle. The point robot is a 2-D double integrator with position and velocity as its state: $\mathbf{x}_k = (x_k, y_k, \dot{x}_k, \dot{y}_k)$. It has a maximum velocity of 2 m/s, and its control input is acceleration up to 1 m/s^2 in any direction. We use these dynamics as in [14], [24] to enable a fair test against other methods that require a robot model. We define a box-shaped safe set (the complement of the obstacle set) as $X_{\text{safe}} = \{\mathbf{x}_k \in X : |x_k| \leq x_{\text{max}}, |y_k| \leq y_{\text{max}}\}$, with $\|(x_{\text{max}}, y_{\text{max}})\|_2 < r$. We use a reward that encourages traveling quickly near the unsafe set: $\rho(\hat{\mathbf{x}}_k, \mathbf{u}_k) = \frac{(\dot{x}_k, \dot{y}_k) \cdot (-y_k, x_k)}{1 + \|(x_k, y_k)\|_2 - r}$.

The hexarotor has the same setup as the quadrotor, but with the addition of wind as an external disturbance. The goal of the hexarotor is to pass through 10 checkpoints in a fixed order while subject to wind (constant speed and direction) and randomly-placed obstacles. Note, offline data collection was performed under wind conditions.

Results and Discussion. The results are summarized in Tables I and II, and in Figure 4. BRSL outperforms the other methods in terms of reward and safety, is not overly conservative, and can operate in real time, despite lacking a model of the robot *a priori*. While SAILR and the baseline RL agent achieved higher speeds, both experienced collisions, unlike BRSL, RTS, and SECAS. In contrast to RTS, which chooses from a low-dimensional parameterized plans, BRSL outputs a more flexible sequence of actions. Furthermore RTS’ planning

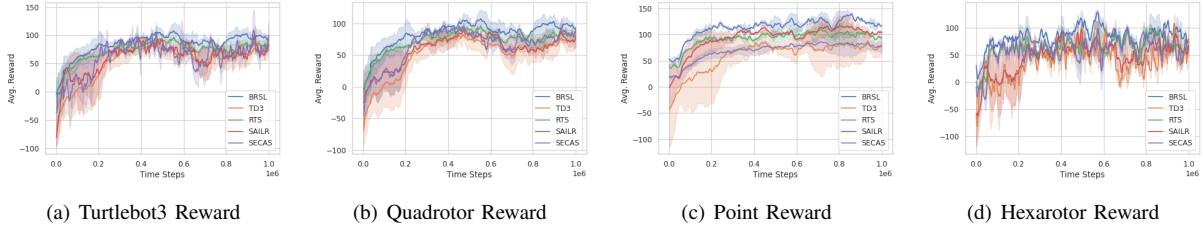


Fig. 4: Average reward over time of BRSL, RTS [28], SAILR [24], and a vanilla TD3 baseline for each of our experiments.

TABLE I: Goal-based experiment results (best values in bold)

	Turtlebot					Quadrotor				
	BRSL	RTS	SAILR	SECAS	Baseline	BRSL	RTS	SAILR	SECAS	Baseline
Goal Rate [%]	57	52	48	53	42	76	66	61	59	54
Collision Rate [%]	0.0	0.0	7.3	0.0	48	0.0	0.0	9.2	0.0	59
Mean/Max Speed [m/s]	.07 / 0.18	0.05 / 0.15	.07 / 0.17	.06 / 0.15	.08 / .18	3.6 / 7.9	3.3 / 7.8	3.7 / 7.9	3.2 / 7.8	3.7 / 7.9
Mean Reward	86	78	68	66	63	82	73	61	68	57
Mean $\pm$ Std. Dev.	50.41 $\pm$ 20.5	100.74 $\pm$ 60.5	30.53 $\pm$ 10.8	22.4 $\pm$ 14.66	10.21 $\pm$ 10.05	60.33 $\pm$ 20.34	260.85 $\pm$ 140.67	45.31 $\pm$ 20.84	39.7 $\pm$ 34.08	20.63 $\pm$ 30.2
Compute Time [ms]										

TABLE II: Path Following Results (best values in bold)

	Point Environment					Hexarotor				
	BRSL	RTS	SAILR	SECAS	Baseline	BRSL	RTS	SAILR	SECAS	Baseline
Collisions [%]	0.0	0.0	4.9	0.0	11.4	0.0	2	14	7	63
Mean Speed [m/s]	0.76	0.72	0.78	0.68	0.86	3.81	3.4	3.84	3.1	3.94
Max Speed [m/s]	2.00	1.89	2.00	1.74	2.00	8.4	8.1	8.2	7.95	8.7
Mean Reward	118	93	88	78	73	84	78	69	62	57
Compute Time [ms]	30.0 $\pm$ 10.2	60.18 $\pm$ 20.09	20.49 $\pm$ 10.34	16.42 $\pm$ 8.7	8.64 $\pm$ 1.14	71.93 $\pm$ 34.52	290.96 $\pm$ 157.24	67.86 $\pm$ 22.34	48.53 $\pm$ 28.69	32.79 $\pm$ 27.6

time increases with state space dimension due to computing a halfspace representation of reachable set zonotopes, which grows exponentially in the number of generators [40]. BRSL avoids this computation by using (6). Instead, the quantity of data for BRSL determines the computation time of M_j from Algorithm 2, used for reachability and adjusting unsafe actions. Therefore, one can ensure the amount of data allows real time operation; choosing the data optimally is left to future work. Finally, BRSL uses zonotopes to exactly represent safety constraints, whereas SECAS uses a more conservative first-order approximation. This results in BRSL achieving higher reward with slightly slower computation time (but still fast enough for real time operation).

V. CONCLUSION

This paper proposes the Black-box Reachability Safety Layer, or BRSL, for safe RL without having a system model *a priori*. BRSL ensures safety via data-driven reachability analysis and a novel technique to push reachable sets out of collision. To enable the RL agent to make dynamics-informed decisions, BRSL also learns an environment model online, which does not affect the safety guarantee. The framework was evaluated on four robot motion planning problems, wherein BRSL respects safety constraints while achieving a high reward over time in comparison to state-of-the-art methods. For future work, we will explore continuous-time settings, reducing the conservativeness of our reachability analysis, and minimizing the amount of data needed to guarantee safety.

REFERENCES

- [1] R. S. Sutton, A. G. Barto *et al.*, *Introduction to reinforcement learning*. MIT press Cambridge, 1998, vol. 135.
- [2] O. Mihatsch and R. Neuneier, “Risk-sensitive reinforcement learning,” *Machine learning*, vol. 49, no. 2, pp. 267–290, 2002.
- [3] J. Garcia and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, no. 1, pp. 1437–1480, 2015.
- [4] P. Geibel and F. Wysotzki, “Risk-sensitive reinforcement learning applied to control under constraints,” *Journal of Artificial Intelligence Research*, vol. 24, pp. 81–108, 2005.
- [5] T. M. Moldovan and P. Abbeel, “Safe exploration in markov decision processes,” 2012.
- [6] Y. Sui, A. Gotovos, J. Burdick, and A. Krause, “Safe exploration for optimization with gaussian processes,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, F. Bach and D. Blei, Eds., vol. 37. Lille, France: PMLR, 07–09 Jul 2015, pp. 997–1005. [Online]. Available: <https://proceedings.mlr.press/v37/sui15.html>
- [7] E. Altman, *Constrained Markov decision processes: stochastic modeling*. Routledge, 1999.
- [8] V. Borkar, “An actor-critic algorithm for constrained markov decision processes,” *Systems & Control Letters*, vol. 54, no. 3, pp. 207–213, 2005. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167691104001276>
- [9] Y. Chow, M. Ghavamzadeh, L. Janson, and M. Pavone, “Risk-constrained reinforcement learning with percentile risk criteria,” 2017.
- [10] C. Tessler, D. J. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” 2018.
- [11] S. Bohez, A. Abdolmaleki, M. Neunert, J. Buchli, N. Heess, and R. Hadsell, “Value constrained model-free continuous control,” 2019.
- [12] D. Ding, X. Wei, Z. Yang, Z. Wang, and M. R. Jovanović, “Provably efficient safe exploration via primal-dual policy optimization,” 2020.
- [13] Y. Efroni, S. Mannor, and M. Pirotta, “Exploration-exploitation in constrained mdp,” 2020.
- [14] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” 2017.
- [15] H. Bharadhwaj, A. Kumar, N. Rhinehart, S. Levine, F. Shkurti, and A. Garg, “Conservative safety critics for exploration,” 2021.
- [16] C. Gehring and D. Precup, “Smart exploration in reinforcement learning using absolute temporal difference errors,” in *Proceedings of the 2013 international conference on Autonomous agents and multi-agent systems*, 2013, pp. 1037–1044.

- [17] R. Koppejan and S. Whiteson, "Neuroevolutionary reinforcement learning for generalized helicopter control," in *Proceedings of the 11th Annual conference on Genetic and evolutionary computation*, 2009, pp. 145–152.
- [18] P. Abbeel and A. Y. Ng, "Exploration and apprenticeship learning in reinforcement learning," in *Proceedings of the 22nd international conference on Machine learning*, 2005, pp. 1–8.
- [19] T. G. Molnar, R. K. Cosner, A. W. Singletary, W. Ubellacker, and A. D. Ames, "Model-free safety-critical control for robotic systems," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 944–951, 2021.
- [20] P. Liu, D. Tateo, H. B. Ammar, and J. Peters, "Robot reinforcement learning on the constraint manifold," in *Conference on Robot Learning*. PMLR, 2022, pp. 1357–1366.
- [21] Y. Chow, O. Nachum, E. Duenez-Guzman, and M. Ghavamzadeh, "A lyapunov-based approach to safe reinforcement learning," *Advances in neural information processing systems*, vol. 31, 2018.
- [22] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 3387–3395.
- [23] E. Squires, R. Konda, S. Coogan, and M. Egerstedt, "Model free barrier functions via implicit evading maneuvers," 2021. [Online]. Available: <https://arxiv.org/abs/2107.12871>
- [24] N. Wagener, B. Boots, and C.-A. Cheng, "Safe reinforcement learning using advantage-based intervention," *arXiv preprint arXiv:2106.09110*, 2021.
- [25] M. Jankovic, "Robust control barrier functions for constrained stabilization of nonlinear systems," *Automatica*, vol. 96, pp. 359–367, 10 2018.
- [26] P. Seiler, M. Jankovic, and E. Hellstrom, "Control barrier functions with unmodeled dynamics using integral quadratic constraints," 2021. [Online]. Available: <https://arxiv.org/abs/2108.10491>
- [27] H. Krasowski, X. Wang, and M. Althoff, "Safe reinforcement learning for autonomous lane changing using set-based prediction," in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2020, pp. 1–7.
- [28] Y. S. Shao, C. Chen, S. Kousik, and R. Vasudevan, "Reachability-based trajectory safeguard (rts): A safe and fast reinforcement learning safety layer for continuous control," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3663–3670, 2021.
- [29] T. Lew, A. Sharma, J. Harrison, A. Byland, and M. Pavone, "Safe active dynamics learning and control: A sequential exploration-exploitation framework," *IEEE Transactions on Robotics*, 2022, in Press.
- [30] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, "Safe exploration in continuous action spaces," *arXiv preprint arXiv:1801.08757*, 2018.
- [31] S. M. LaValle, *Planning algorithms*. Cambridge university press, 2006.
- [32] S. Kousik, S. Vaskov, F. Bu, M. Johnson-Roberson, and R. Vasudevan, "Bridging the gap between safety and real-time performance in receding-horizon trajectory design for mobile robots," *The International Journal of Robotics Research*, vol. 39, no. 12, pp. 1419–1469, 2020.
- [33] K. Leung, E. Schmerling, M. Zhang, M. Chen, J. Talbot, J. C. Gerdes, and M. Pavone, "On infusing reachability-based safety assurance within planning frameworks for human–robot vehicle interactions," *The International Journal of Robotics Research*, vol. 39, no. 10-11, pp. 1326–1345, 2020.
- [34] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," 2016.
- [35] Y. F. Chen, M. Everett, M. Liu, and J. P. How, "Socially aware motion planning with deep reinforcement learning," 2018.
- [36] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," 2018.
- [37] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 31–36.
- [38] L. Tai, J. Zhang, M. Liu, and W. Burgard, "Socially compliant navigation through raw depth inputs with generative adversarial imitation learning," 2018.
- [39] Y. Ohta, H. Maeda, and S. Kodama, "Reachability, observability, and realizability of continuous-time positive systems," *SIAM journal on control and optimization*, vol. 22, no. 2, pp. 171–180, 1984.
- [40] M. Althoff, "Reachability analysis and its application to the safety assessment of autonomous cars," Ph.D. dissertation, Technische Universität München, 07 2010.
- [41] T. Koller, F. Berkenkamp, M. Turchetta, and A. Krause, "Learning-based model predictive control for safe exploration," in *2018 IEEE conference on decision and control (CDC)*. IEEE, 2018, pp. 6059–6066.
- [42] A. Alanwar, A. Koch, F. Allgöwer, and K. H. Johansson, "Data-driven reachability analysis using matrix zonotopes," in *Learning for Dynamics and Control*. PMLR, 2021, pp. 163–175.
- [43] J. K. Scott, D. M. Raimondo, G. R. Marseglia, and R. D. Braatz, "Constrained zonotopes: A new tool for set-based estimation and fault detection," *Automatica*, vol. 69, pp. 126–136, 2016.
- [44] V. Raghuraman and J. P. Koeln, "Set operations and order reductions for constrained zonotopes," *arXiv preprint arXiv:2009.06039*, 2020.
- [45] S. Magdici and M. Althoff, "Fail-safe motion planning of autonomous vehicles," in *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2016, pp. 452–458.
- [46] D. Fridovich-Keil, J. F. Fisac, and C. J. Tomlin, "Safely probabilistically complete real-time planning and exploration in unknown environments," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 7470–7476.
- [47] D. Althoff, M. Althoff, and S. Scherer, "Online safety verification of trajectories for unmanned flight with offline computed robust invariant sets," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2015, pp. 3470–3477.
- [48] A. Shetty and G. X. Gao, "Predicting state uncertainty bounds using non-linear stochastic reachability analysis for urban gnss-based uas navigation," *IEEE Transactions on Intelligent Transportation Systems*, 2020.
- [49] S. Vaskov, H. Larson, S. Kousik, M. Johnson-Roberson, and R. Vasudevan, "Not-at-fault driving in traffic: A reachability-based approach," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2019, pp. 2785–2790.
- [50] B. Amos and J. Z. Kolter, "Optnet: Differentiable optimization as a layer in neural networks," in *International Conference on Machine Learning*. PMLR, 2017, pp. 136–145.
- [51] L. K. Chung, A. Dai, D. Knowles, S. Kousik, and G. X. Gao, "Constrained feedforward neural network training via reachability analysis," 2021.
- [52] S. Dankwa and W. Zheng, "Twin-delayed ddpg: A deep reinforcement learning technique to model a continuous movement of an intelligent robot agent," in *Proceedings of the 3rd International Conference on Vision, Image and Signal Processing*, 2019, pp. 1–5.
- [53] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," 2018.
- [54] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2019.
- [55] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," *Advances in neural information processing systems*, vol. 31, 2018.